

PowerPuffGenes Report

2025-03-17

Contents

Introduction	1
RNA sequence analysis	1
nf-core/rnaseq Pipeline	1
Differential Expression Analysis	2
Processing and analysis of count and tpm data	2
Finding Prolonged Genes	13
Heatmaps	17
Circos plot	20
Results	25
IGV	25
Bibliography	29

Introduction

Understanding the temporal dynamics of gene expression is crucial for deciphering complex biological processes like stem cell differentiation or response to stimuli. This project utilizes a doxycycline (dox)-inducible system in mouse stem cells, where dox triggers gene expression via the rtTA-TRE mechanism. While powerful, concerns exist that dox itself might exert off-target effects on the transcriptome. Leveraging Dr. Rinn's lab RNA-seq dataset spanning five time points (0, 12, 24, 48, 96 hours) with replicates, this analysis aims to: 1) Identify genes significantly regulated following dox administration. 2) Characterize the temporal profiles of these regulated genes, distinguishing between transient and sustained responses. 3) Investigate the role, significance, and chromosomal location of these genes. We employ the nf-core/rnaseq pipeline for data processing and DESeq2 for differential expression analysis, providing a practical application of bioinformatics skills to time-course data exploration.

RNA sequence analysis

nf-core/rnaseq Pipeline

The raw sequencing reads were processed using the nf-core/rnaseq pipeline version 3.14, a standardized and reproducible bioinformatics pipeline built using Nextflow.

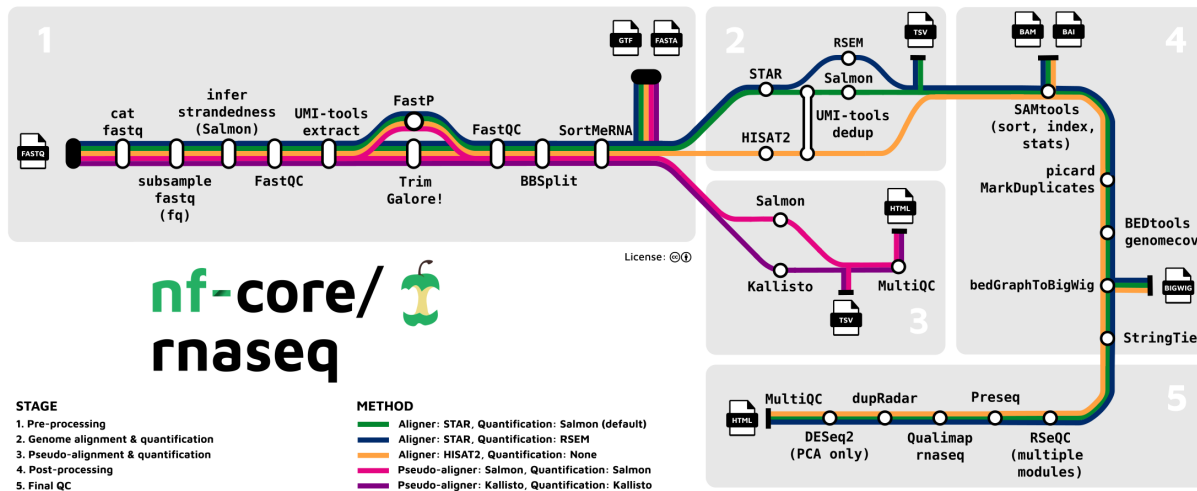


Figure 1: nf-core/rnaseq diagram

In brief, the pipeline performs the following key steps: 1. **Quality Control:** Raw reads were assessed using FastQC and summarised using MultiQC. 2. **Read Alignment:** Reads were aligned to the mouse reference genome (e.g., GRCm39/mm39, specify version) using the STAR aligner. STAR (Spliced Transcripts Alignment to a Reference) is efficient at mapping RNA-seq reads across splice junctions. 3. **Quantification:** Gene expression levels were quantified using Salmon (or RSEM, specify which was used by the pipeline run). Salmon is a fast tool for transcript quantification using quasi-mapping and a lightweight alignment approach, which then summarizes transcript counts to the gene level.

Key Input/Output Files

- **Input:** Raw FASTQ files (paired-end reads recommended) and a sample sheet defining the experimental design (samples, conditions, replicates).
- **Output:** The pipeline generates numerous output files. Key files used in this analysis include:
 - `salmon.merged.gene_counts.tsv`: Raw gene-level counts estimated by Salmon. Used as input for DESeq2 statistical modeling.
 - `salmon.merged.gene_tpm.tsv`: Gene-level Transcripts Per Million (TPM) values. Used for expression level interpretation and visualization, as TPM is normalized for gene length and sequencing depth.
 - `multiqc/multiqc_report.html`: An aggregated quality control report across all samples.
 - Alignment files (BAM format): Detailed mapping results (not directly used in this DE analysis but available).

Differential Expression Analysis

Processing and analysis of count and tpm data

Parameters and paths definition

```
COUNTS_PATH <- "data/NF3-18/salmon.merged.gene_counts.tsv"
TPM_PATH    <- "data/NF3-18/salmon.merged.gene_tpm.tsv"
```

```

FILTER_TRESHOLD <- 0
P_VALUE_FILTER  <- 0.05
LOG2_FOLD_FILTER <- 1

```

Loading and preprocessing count and tpm files

```

# Read raw data and make dictionary mapping gene id with gene name
counts_orig <- read.table(COUNTS_PATH, header=TRUE, row.names=1)
tpm_orig    <- read.table(TPM_PATH, header=TRUE, row.names=1)
g2s         <- data.frame(gene_id = rownames(counts_orig),
                          gene_name = counts_orig[, 1])

# Remove gene name column from counts and convert counts to numerical rounded matrix
counts      <- counts_orig %>% select(-gene_name)
tpm         <- tpm_orig %>% select(-gene_name)
counts_matrix <- as.matrix(counts)
counts_matrix <- round(counts_matrix)

# Filter genes with no counts for any of the samples
counts_filtered <- counts_matrix[rowSums(counts_matrix) > FILTER_TRESHOLD, ]
tpm_filtered    <- tpm[rowSums(tpm) > FILTER_TRESHOLD, ]

save(counts, tpm, counts_filtered, tpm_filtered, g2s, file = "results/data_files.RData")

```

DESeq2 analysis for counts

```

# Make a column from the titles of the columns of the counts_matrix table
deseq_samples <- data.frame(sample_id = colnames(counts))

# Get the time and replicate values from the sample names in deseq_samples
split_values  <- strsplit(deseq_samples$sample_id, "_")
time_values   <- sapply(split_values, function(x) x[[2]])
replicate_values <- sapply(split_values, function(x) x[[3]])

# Add time and replicate values as columns to deseq_samples and factor them
deseq_samples$time_point <- time_values
deseq_samples$replicate  <- replicate_values
deseq_samples$time_point <- factor(deseq_samples$time_point, levels=c("0", "12", "24", "48", "96"))
deseq_samples$replicate  <- factor(deseq_samples$replicate)

# Testing whether sample sheet and counts are arranged properly
stopifnot(all(colnames(counts) == rownames(deseq_samples$sample_id)))

# Prepare DESeq dataset and run DESeq2
dds <- DESeqDataSetFromMatrix(countData = counts_filtered,
                              colData = deseq_samples,
                              design = ~ time_point)

dds <- DESeq(dds)

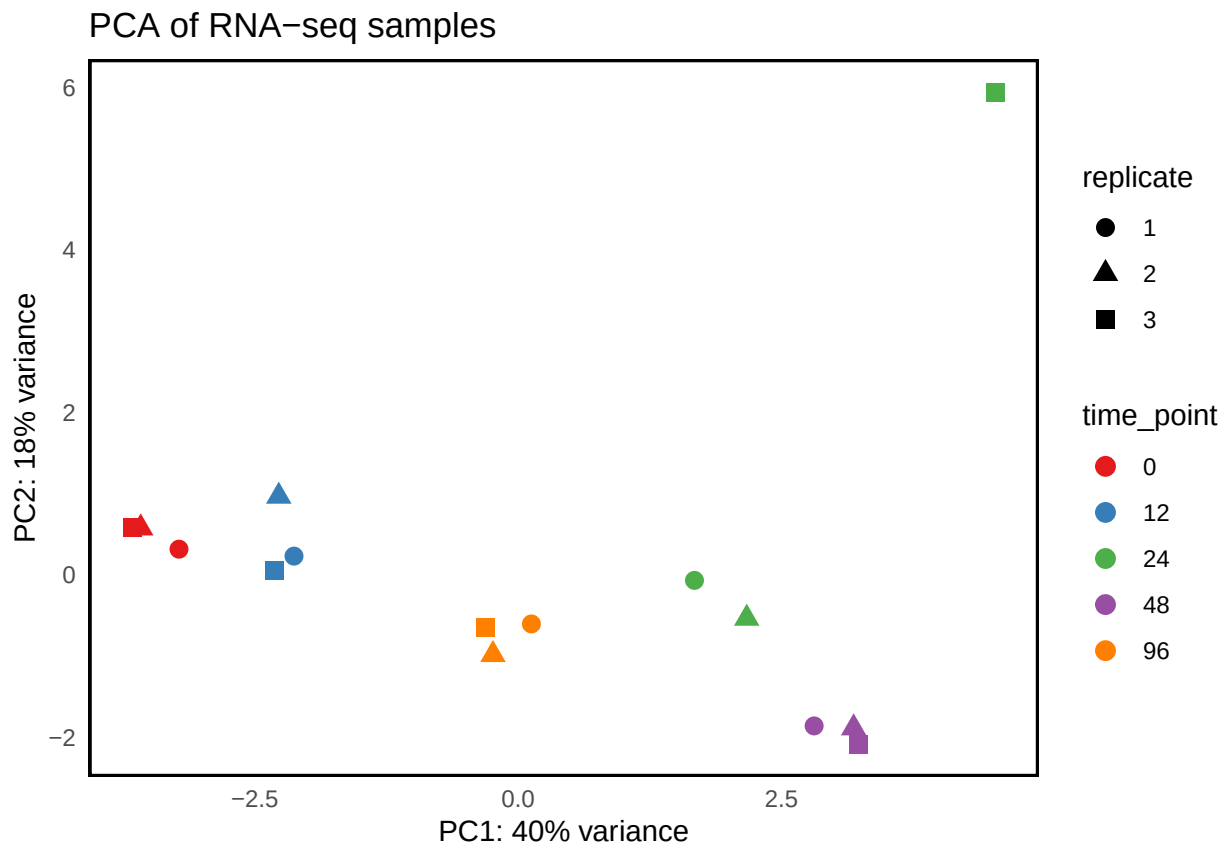
```

```

# PCA plot function
plot_pca <- function(dds, intgroup="time_point", ntop=500) {
  vsd <- vst(dds, blind=FALSE) # Get variance stabilized data
  pca_data <- plotPCA(vsd, intgroup=intgroup, returnData=TRUE, ntop=ntop) # Run PCA
  percentVar <- round(100 * attr(pca_data, "percentVar")) # Percentage of variance explained
  # Plot
  ggplot(pca_data, aes(x=PC1, y=PC2, color=time_point, shape=replicate)) +
    geom_point(size=3) +
    xlab(paste0("PC1: ", percentVar[1], "% variance")) +
    ylab(paste0("PC2: ", percentVar[2], "% variance")) +
    ggtitle("PCA of RNA-seq samples") +
    theme_minimal() +
    theme(
      panel.grid.major = element_blank(),
      panel.grid.minor = element_blank(),
      panel.border = element_rect(color = "black", fill = NA, linewidth = 1)
    ) +
    scale_color_brewer(palette = "Set1")
}

pca_plot <- plot_pca(dds, intgroup=c("time_point", "replicate"))
print(pca_plot)

```



```

ggsave("results/pca_plot.png", pca_plot, width = 8, height = 6, dpi = 300)

```


Obtaining and filtering count results

```
# Define the comparisons
comparison_list <- list(
  # Time points against baseline
  time_12_vs_0 = c("time_point", "12", "0"),
  time_24_vs_0 = c("time_point", "24", "0"),
  time_48_vs_0 = c("time_point", "48", "0"),
  time_96_vs_0 = c("time_point", "96", "0"),
  # Time series comparisons
  time_24_vs_12 = c("time_point", "24", "12"),
  time_48_vs_24 = c("time_point", "48", "24"),
  time_96_vs_48 = c("time_point", "96", "48")
)

# Create empty df to store results values (schema looks good)
results <- data.frame("gene_id" = character(),
  "baseMean" = numeric(),
  "log2FoldChange" = numeric(),
  "lfcSE" = numeric(),
  "stat" = numeric(),
  "pvalue" = numeric(),
  "padj" = numeric(),
  "gene_name" = character(),
  "comparison" = character())

# Loop through the defined contrasts
for(i in names(comparison_list)) {
  current_comparison <- comparison_list[[i]] # get comparison i
  res <- results(dds, contrast = current_comparison) # get results for time comparison i using 'contrast'
  if (is.null(res)) {
    warning(paste("Could not generate results for contrast:", contrast_name))
    next}
  # Temporary df to store the results for comparison i
  tmp_res_df <- res %>%
    as.data.frame() %>%
    rownames_to_column("gene_id") %>%
    merge(g2s, by = "gene_id", all.x = TRUE) %>%
    mutate(comparison = i)
  # Add the temporary df to the main results df
  results <- bind_rows(results, tmp_res_df)
}

# Filter based on p-value at log2fold changes defined at top
filtered_results <- results %>%
  filter(padj < P_VALUE_FILTER, abs(log2FoldChange) > LOG2_FOLD_FILTER)

# Get all gene names that are significant (drop gene name repetitions)
filtered_genes <- as.data.frame(filtered_results$gene_name, collapse = "\n")
filtered_genes <- unique(filtered_genes)
colnames(filtered_genes)[1] <- "gene_name"

save(results, filtered_results, filtered_genes, file = "results/results_counts.RData")
```

Obtaining tmp results

```
# Loop to calculate avg and sd for replicates at a given time point
time_points <- c("0", "12", "24", "48", "96")
avg_and_sd_values <- list()
for (tp in time_points) {
  cols <- grep(paste0("WT_", tp, "_"), colnames(tpm_filtered))
  avg <- rowMeans(tpm_filtered[, cols])
  sd <- apply(tpm_filtered[, cols], 1, sd)
  sd <- data.frame(sd)
  combined <- cbind(avg, sd)
  avg_and_sd_values <- c(avg_and_sd_values, list(combined))
}

# Convert the list to a data frame and add column names for the respective timepoint
avg_and_sd_values <- do.call(cbind, avg_and_sd_values)
colnames(avg_and_sd_values) <- paste0(rep(time_points, each = 2), c("_avg", "_sd"))
avg_and_sd_values <- as.data.frame(avg_and_sd_values) %>% rownames_to_column("gene_id") %>% m

save(avg_and_sd_values, file = "results/results_tpm.RData" )
```

Distribution of count results

```
histogram_data <- filtered_results %>%
  select(baseMean, lfcSE, padj, log2FoldChange) %>%
  pivot_longer(
    cols = everything(),
    names_to = "metric",
    values_to = "value"
  ) %>%
  # Convert metric to a factor with a specific order
  mutate(metric = factor(metric, levels = c("baseMean", "lfcSE", "padj", "log2FoldChange"),
    labels = c("Base Mean", "LFC Standard Error",
      "Adjusted P-value", "Log2 Fold Change")))

# Create a function to set appropriate limits for each metric
get_limits <- function(metric_name) {
  switch(as.character(metric_name),
    "Base Mean" = c(0, 500),
    "LFC Standard Error" = c(0, 2),
    "Adjusted P-value" = c(0, P_VALUE_FILTER),
    "Log2 Fold Change" = c(-5, 5))
}

# Create a nice color palette
histogram_colors <- c("Base Mean" = "#3498db",
  "LFC Standard Error" = "#2ecc71",
  "Adjusted P-value" = "#e74c3c",
  "Log2 Fold Change" = "#f1c40f")

# Create faceted plot with different x-axis limits for each facet
```

```

faceted_plot <- ggplot(histogram_data, aes(x = value, fill = metric)) +
  geom_histogram(bins = 50, color = "white", alpha = 0.85) +
  facet_wrap(~ metric, scales = "free", ncol = 2) +
  scale_fill_manual(values = histogram_colors) +
  labs(
    title = "Distribution of DESeq2 Statistics",
    x = NULL, # x-axis label is redundant with facet titles
    y = "Count"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    strip.background = element_rect(fill = "gray95"),
    strip.text = element_text(face = "bold", size = 12),
    plot.title = element_text(face = "bold", hjust = 0.5, size = 16),
    panel.grid.minor = element_blank(),
    panel.spacing = unit(1, "lines"),
    legend.position = "none" # Hide the legend as it's redundant
  )

# Save high-resolution version
ggsave("results/counts_distributions.png", faceted_plot,
       width = 10, height = 8, dpi = 300)

```

Distribution of tpm results

```

# Make a distribution of TPM values in each time point - using facet-wrap
time_values <- names(avg_and_sd_values)[grep("avg", names(avg_and_sd_values))]
melt_tpm_df <- melt(avg_and_sd_values, measure.vars = time_values, value_name = "gene_id")
ggplot(melt_tpm_df, aes(x = log2(value))) +
  geom_histogram(bins = 50) +
  facet_wrap(~ variable, scales = "free_x") +
  theme("paperwhite")

```

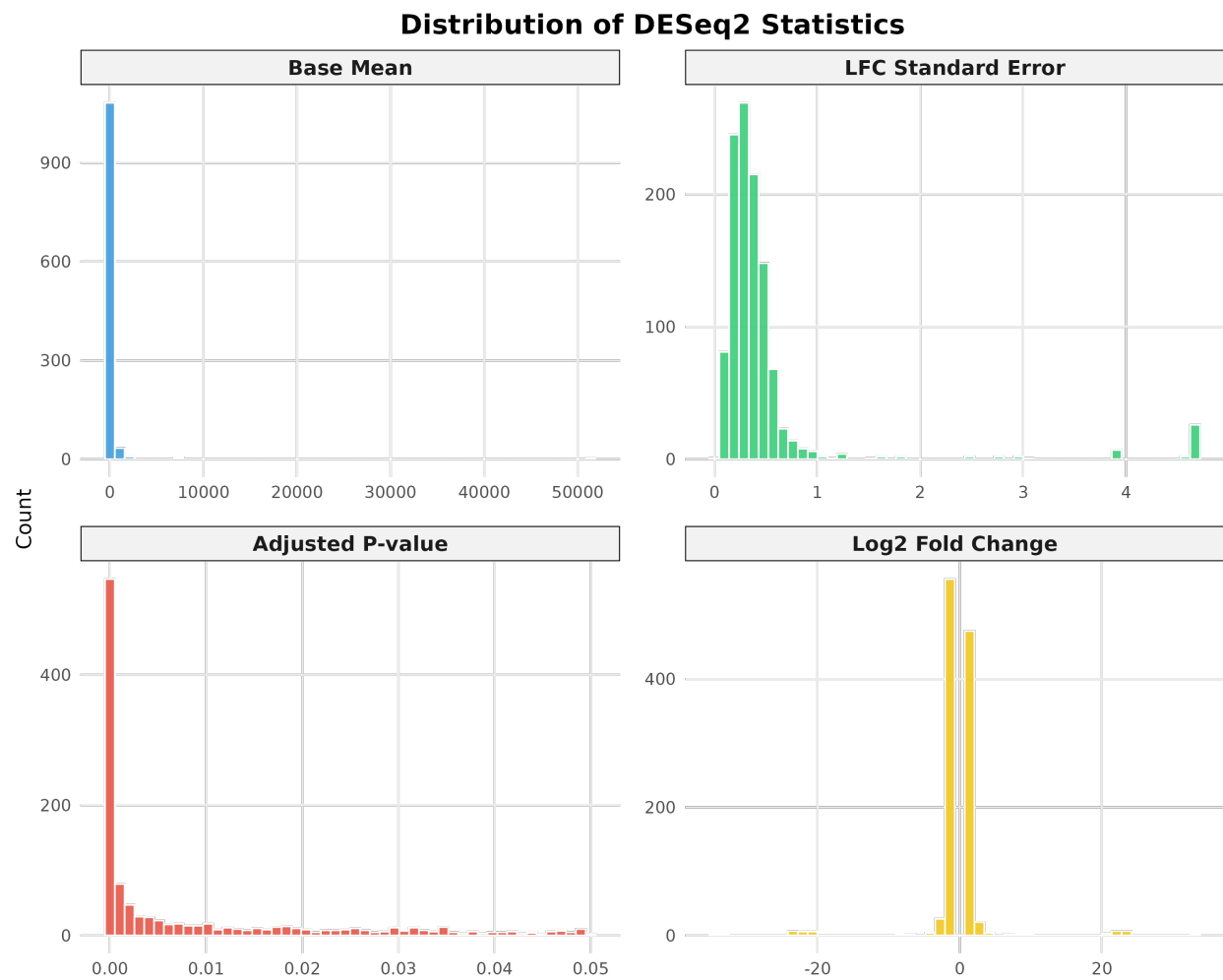
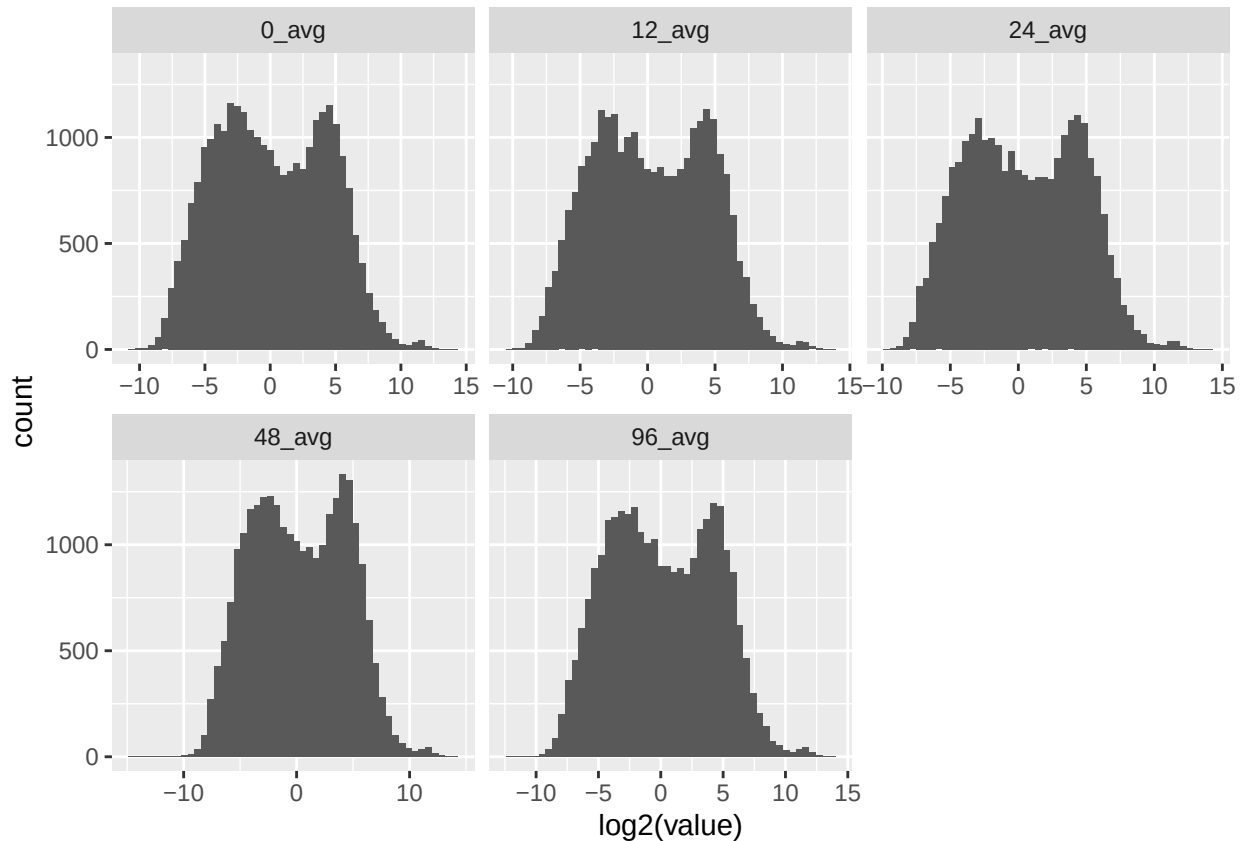


Figure 2: Count Distribution



Volcano plots

```
# Pre-process results for volcano plot
filtered_results_vp <- results %>%
  mutate(
    negLog10Padj = -log10(padj),
    significance = case_when(
      padj < P_VALUE_FILTER & log2FoldChange > LOG2_FOLD_FILTER ~ "Upregulated",
      padj < P_VALUE_FILTER & log2FoldChange < -LOG2_FOLD_FILTER ~ "Downregulated",
      TRUE ~ "Not Significant" ),
    significance = factor(significance, levels = c("Upregulated", "Downregulated", "Not Significant")),
    gene_label = if_else(significance != "Not Significant", gene_name, "")
  ) %>%
  filter(!is.na(padj) & !is.na(log2FoldChange))

# Define color map and theme
color_map <- c("Upregulated" = "red", "Downregulated" = "blue", "Not Significant" = "grey")
common_volcano_theme <- theme_minimal(base_size = 10) +
  theme(
    legend.position = "bottom",
    plot.title = element_text(hjust = 0.5, face = "bold", size = rel(1.2)),
    strip.text = element_text(face = "bold", size=rel(0.9)), # Style facet titles
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
```

```

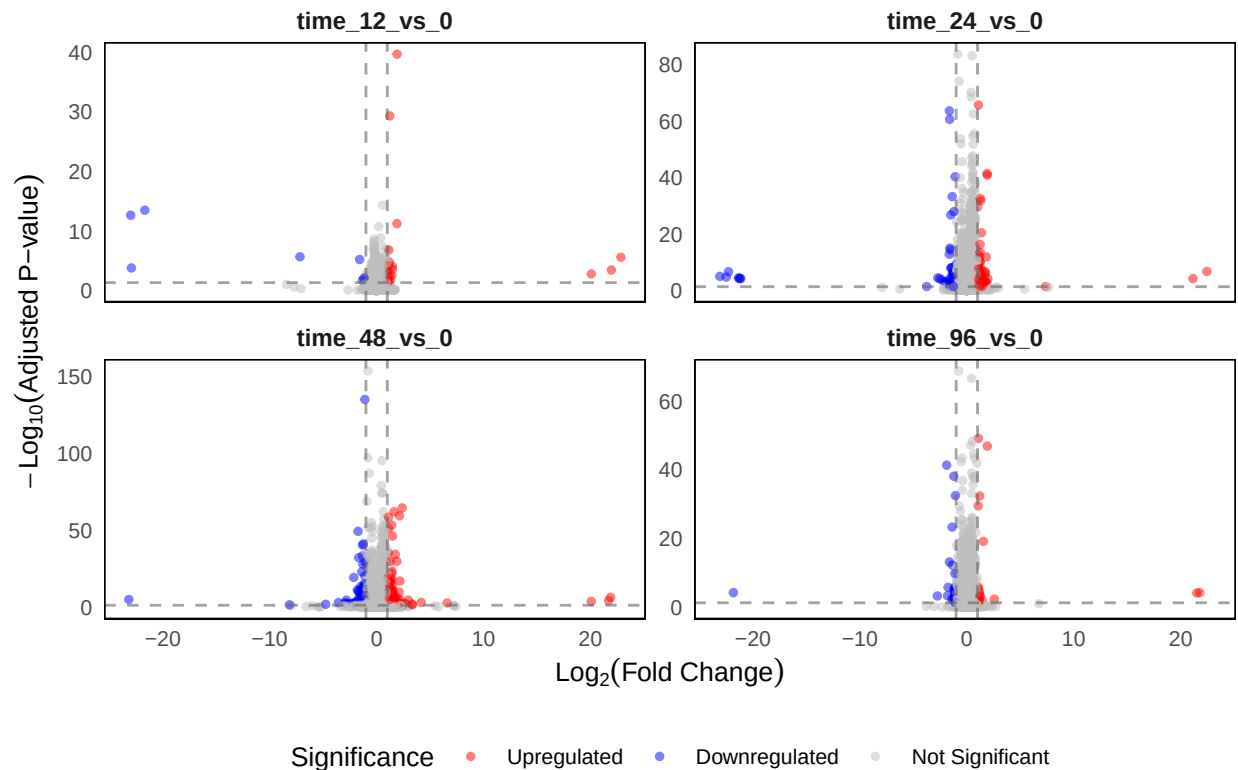
    panel.border = element_rect(color = "black", fill = NA, size = 0.5)
  )

# Filter Baseline Comparisons
baseline_comparisons <- c("time_12_vs_0", "time_24_vs_0", "time_48_vs_0", "time_96_vs_0")
filtered_results_vp_baseline<- filtered_results_vp %>%
  filter(comparison %in% baseline_comparisons) %>%
  mutate(comparison = factor(comparison, levels = baseline_comparisons))

# Plot Baseline Comparisons
plot_baseline <- ggplot(
  filtered_results_vp_baseline,
  aes(x = log2FoldChange, y = negLog10Padj, color = significance, label = gene_label)
) +
  geom_point(alpha = 0.5, size = 1.0) +
  scale_color_manual(values = color_map, name = "Significance") +
  geom_vline(xintercept = c(-LOG2_FOLD_FILTER, LOG2_FOLD_FILTER), lty = "dashed", color = "grey40", alpha=0.6) +
  geom_hline(yintercept = -log10(P_VALUE_FILTER), lty = "dashed", color = "grey40", alpha=0.6) +
  # Facet into a 2x2 grid
  facet_wrap(~ comparison, ncol = 2, scales = "free_y") +
  labs(
    title = "Volcano Plots: Comparisons vs Baseline",
    x = expression(Log2("Fold Change")),
    y = expression(-Log10("Adjusted P-value"))
  ) +
  common_volcano_theme +
  coord_cartesian(ylim = c(0, NA))
print(plot_baseline)

```

Volcano Plots: Comparisons vs Baseline



```
ggsave("results/volcano_baseline.png", width = 20, height = 12)

# Filter Time Series Comparisons
timeseries_comparisons <- c("time_12_vs_0", "time_24_vs_12", "time_48_vs_24", "time_96_vs_48")
filtered_results_vp_timeseries <- filtered_results_vp %>%
  filter(comparison %in% timeseries_comparisons) %>%
  # Ensure 'comparison' factor levels are only those present, in the desired order
  mutate(comparison = factor(comparison, levels = timeseries_comparisons))

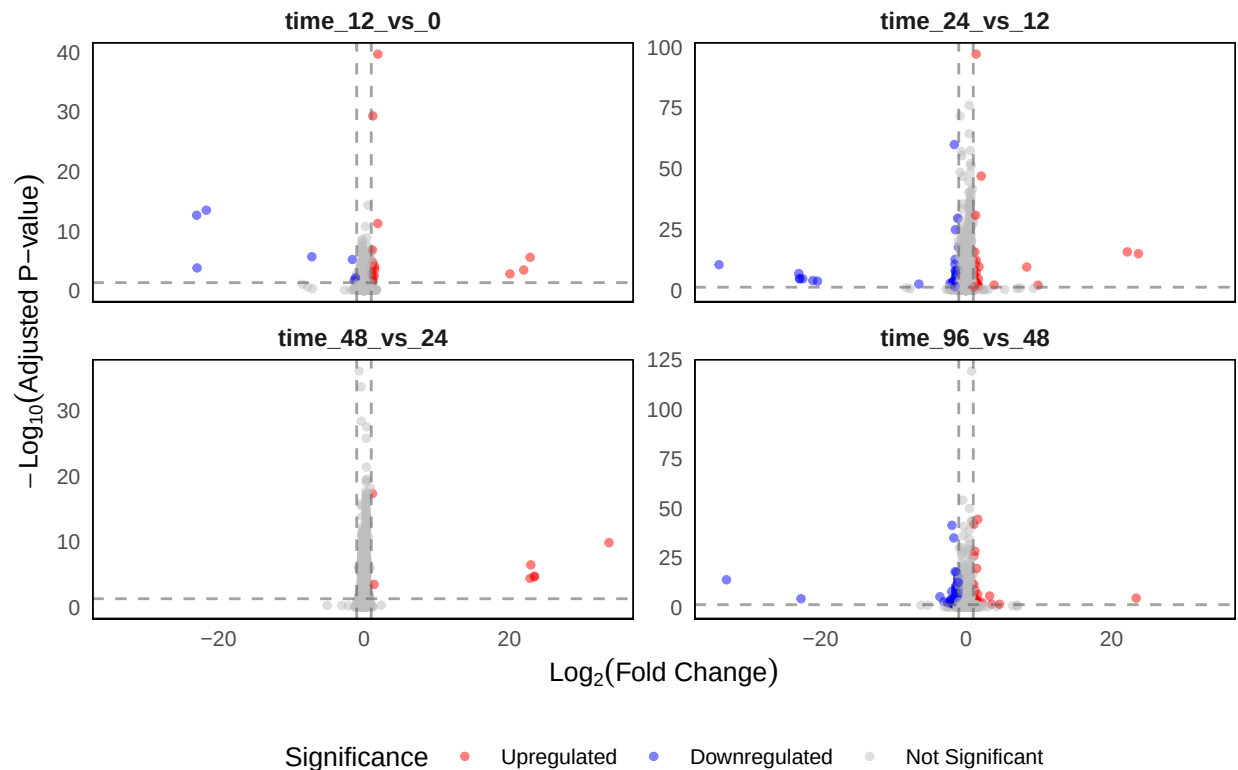
# Plot Time Series Comparisons
plot_timeseries <- ggplot(
  filtered_results_vp_timeseries,
  aes(x = log2FoldChange, y = negLog10Padj, color = significance, label = gene_label)
) +
  geom_point(alpha = 0.5, size = 1.0) +
  scale_color_manual(values = color_map, name = "Significance") +
  geom_vline(xintercept = c(-LOG2_FOLD_FILTER, LOG2_FOLD_FILTER), lty = "dashed", color = "grey40", alpha = 0.6) +
  geom_hline(yintercept = -log10(P_VALUE_FILTER), lty = "dashed", color = "grey40", alpha = 0.6) +
  # Facet into a 2x2 grid
  facet_wrap(~ comparison, ncol = 2, scales = "free_y") +
  labs(
    title = "Volcano Plots: Sequential Time Comparisons",
    x = expression(Log[2]("Fold Change")),
    y = expression(-Log[10]("Adjusted P-value"))
  ) +
  common_volcano_theme
```

```
coord_cartesian(ylim = c(0, NA))
```

```
## <ggproto object: Class CoordCartesian, Coord, gg>
##   aspect: function
##   backtransform_range: function
##   clip: on
##   default: FALSE
##   distance: function
##   expand: TRUE
##   is_free: function
##   is_linear: function
##   labels: function
##   limits: list
##   modify_scales: function
##   range: function
##   render_axis_h: function
##   render_axis_v: function
##   render_bg: function
##   render_fg: function
##   setup_data: function
##   setup_layout: function
##   setup_panel_guides: function
##   setup_panel_params: function
##   setup_params: function
##   train_panel_guides: function
##   transform: function
##   super: <ggproto object: Class CoordCartesian, Coord, gg>
```

```
print(plot_timeseries)
```


Volcano Plots: Sequential Time Comparisons



```
ggsave("results/volcano_timeseries.png", width = 20, height = 12)
```

Finding Prolonged Genes

```
filtered_vs_zero <- filtered_results[grep("vs_0", filtered_results$comparison), ]

# filter genes that are significantly changed at any time point and end time point (prolonged)
prolonged_res <- filtered_vs_zero %>% group_by(gene_id) %>% filter(n()>1)
prolonged_genes <- prolonged_res[grep("96_vs", prolonged_res$comparison),
                                   which(names(prolonged_res)=="gene_name")]
prolonged_res_results <- merge(filtered_vs_zero, prolonged_genes)

# group prolonged genes by their triggering time points
change_at_12 <- prolonged_res_results[grep("12_vs", prolonged_res_results$comparison),
                                       which(names(prolonged_res_results)=="gene_name")]
change_at_24 <- prolonged_res_results[grep("24_vs", prolonged_res_results$comparison),
                                       which(names(prolonged_res_results)=="gene_name")]
change_at_48 <- prolonged_res_results[grep("48_vs", prolonged_res_results$comparison),
                                       which(names(prolonged_res_results)=="gene_name")]
change_at_12_to_end <- Reduce(intersect, list(change_at_12, change_at_24, change_at_48))
change_at_24_to_end <- intersect(change_at_24, change_at_48)
change_at_24_to_end <- change_at_24_to_end[!(change_at_24_to_end %in% change_at_12_to_end)]
change_at_48_to_end <- change_at_48[!(change_at_48 %in% change_at_12_to_end) &
```

```

!(change_at_48 %in% change_at_24_to_end)]

# filter genes that are significantly changed at one time point (transient)
transient_genes <- filtered_vs_zero[!(filtered_vs_zero$gene_name %in% prolonged_genes$gene_name),
                                     which(names(filtered_vs_zero)=="gene_name")]
# Note: All transient genes are changed only at a particular time point (i.e., spike).
save(change_at_12_to_end, change_at_24_to_end, change_at_48_to_end, transient_genes,
      file="results/gene_groups.RData")

```

LFC plot

```

# set of genes which prolonged change in the expression after 12 hours until the end.
genes_of_interest <- data.frame(
  gene_name = c(change_at_12_to_end, change_at_24_to_end, change_at_48_to_end),
  change_from = c(rep("12", length(change_at_12_to_end)),
                  rep("24", length(change_at_24_to_end)),
                  rep("48", length(change_at_48_to_end)))
)

res_df = merge(results, genes_of_interest)

df_time0 <- res_df %>% filter(grepl("_vs_0$", comparison))
df_intermediate <- res_df %>% filter(!grepl("_vs_0$", comparison))

df_time0 <- df_time0 %>%
  mutate(Time = as.numeric(gsub("time_([0-9]+)_vs_0", "\\1", comparison)))

df_intermediate <- df_intermediate %>%
  mutate(Time1 = as.numeric(gsub("time_([0-9]+)_vs_([0-9]+)", "\\1", comparison)),
         Time2 = as.numeric(gsub("time_([0-9]+)_vs_([0-9]+)", "\\2", comparison)))

df_intermediate <- df_intermediate %>%
  group_by(gene_name) %>%
  mutate(Offset_Index = row_number() - 1) %>%
  ungroup()

# Calculate y-axis limits for each gene
y_axis_limits <- res_df %>%
  group_by(gene_name) %>%
  summarize(
    min_log2FC = min(log2FoldChange, na.rm = TRUE),
    max_log2FC = max(log2FoldChange, na.rm = TRUE),
    y_axis_limit = max_log2FC + (max_log2FC - min_log2FC) * 0.05 # Add 10% padding
  )

df_intermediate <- df_intermediate %>%
  left_join(y_axis_limits, by = "gene_name") %>%
  mutate(
    #Offset_Index = row_number(), # Assign each comparison a unique index
    Y_Pos = y_axis_limit + (Offset_Index * 0.3) # Stagger p-value markers above max limit
  )

```

```

# Define significance levels for p-value markers
df_intermediate <- df_intermediate %>%
  mutate(Significance = case_when(
    pvalue < 0.001 ~ "***",
    pvalue < 0.01 ~ "**",
    pvalue < 0.05 ~ "*",
    TRUE ~ ""
  ))

# Create the plot with facets for each gene
prolonged_lfc <- ggplot() +
  # Plot log2FoldChange for time 0 comparisons
  geom_point(data = df_time0, aes(x = Time, y = log2FoldChange, color = padj < 0.05), size = 1.5) +
  geom_line(data = df_time0, aes(x = Time, y = log2FoldChange, group = gene_name)) +

  # Add p-value markers as separate, non-linking lines (for all comparisons)
  geom_segment(data = df_intermediate %>% filter(Significance != ""),
    aes(x = Time1, xend = Time2,
        y = Y_Pos, yend = Y_Pos),
    color = "black", size = 0.5) +

  # Add significance markers (***, **, *) only for significant comparisons
  geom_text(data = df_intermediate %>% filter(Significance != ""),
    aes(x = (Time1 + Time2) / 2, y = Y_Pos + 0.025, label = Significance),
    size = 4, color = "black") +

  # Facet each gene into separate plots
  facet_wrap(~gene_name, scales = "free") +
  coord_cartesian() +

  # Customize x-axis ticks to show only specific time points
  scale_x_continuous(breaks = c(12, 24, 48, 96), labels = c("12", "24", "48", "96")) +

  # Labels and formatting
  labs(title = "Genes regulated by dox over time",
    caption = " *: p < 0.05\n **: p < 0.01\n ***: p < 0.001",
    x = "Time after dox induction (hours)",
    y = "log2fold change",
    color = "Significant (padj < 0.05)\n compared to the zero timepoint") +
  theme_minimal() +
  theme(
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.background = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, size = 1)
  )
ggsave("results/lfc_prolonged_genes.png", width = 20, height = 12, dpi = 500)

```

TPM plot

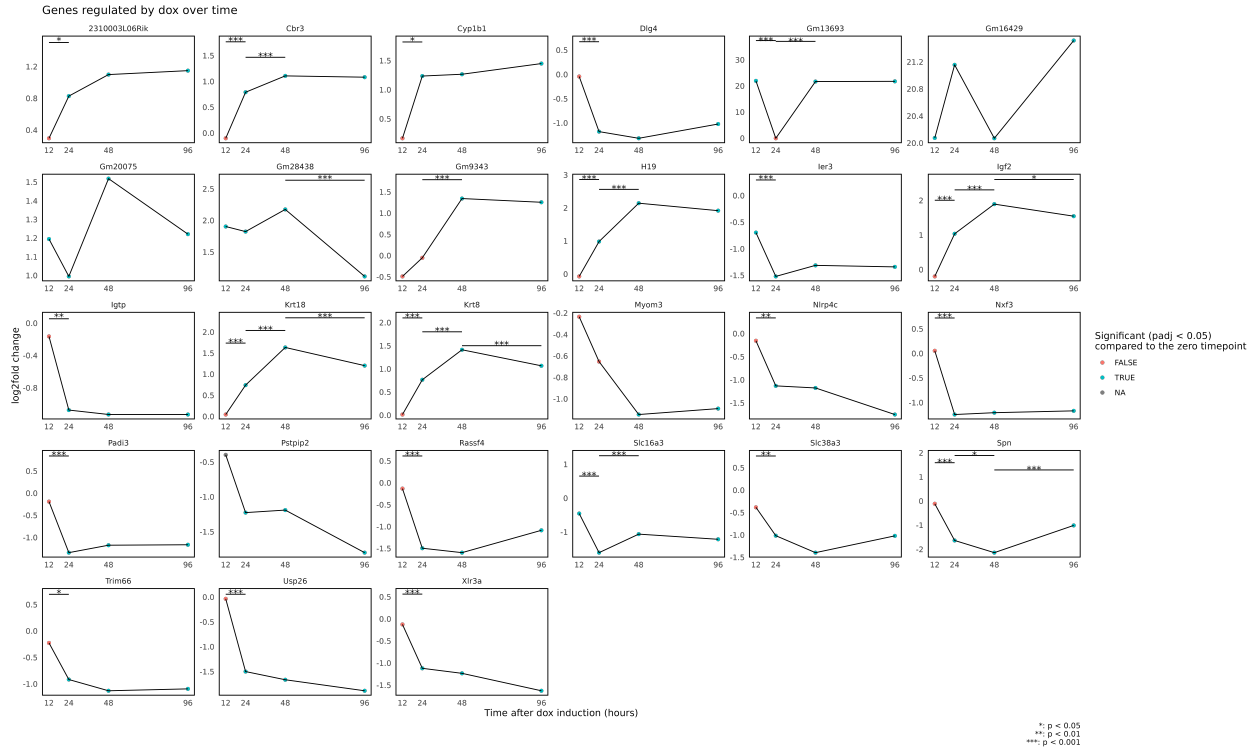


Figure 3: lfc_prolonged_genes

```
sig_gene <- as.data.frame(unique(filtered_results$gene_id))
names(sig_gene) <- "gene_id"

# subset the data frame of avg / sd TPM values
sig_gene_tpm <- inner_join(avg_and_sd_values, sig_gene, by = "gene_id")
sig_gene_tpm <- merge(sig_gene_tpm, g2s)

# prepare TPM data for plotting genes of interest
df_plot_genes <- merge(sig_gene_tpm, genes_of_interest)

time_avg_values <- names(df_plot_genes)[grep("avg", names(df_plot_genes))]
df_plot_genes_avg <- melt(df_plot_genes, measure.vars = time_avg_values, value.name = "gene_id")
df_plot_genes_avg$Time <- do.call(rbind, stringr::str_split(df_plot_genes_avg$variable, '_avg')[,1])
df_plot_genes_avg$Time <- as.numeric(df_plot_genes_avg$Time)
colnames(df_plot_genes_avg)[which(colnames(df_plot_genes_avg) == 'value')] <- 'avg'
df_plot_genes_avg <- df_plot_genes_avg %>% select("gene_name", "Time", "avg")

time_sd_values <- names(df_plot_genes)[grep("sd", names(df_plot_genes))]
df_plot_genes_sd <- melt(df_plot_genes, measure.vars = time_sd_values, value.name = "gene_id")
df_plot_genes_sd$Time <- do.call(rbind, stringr::str_split(df_plot_genes_sd$variable, '_sd')[,1])
df_plot_genes_sd$Time <- as.numeric(df_plot_genes_sd$Time)
colnames(df_plot_genes_sd)[which(colnames(df_plot_genes_sd) == 'value')] <- 'sd'
df_plot_genes_sd <- df_plot_genes_sd %>% select("gene_name", "Time", "sd")

df_plot_genes_avg_sd <- merge(df_plot_genes_avg, df_plot_genes_sd, by = c("gene_name", "Time"))
```

```

prolonged_tpm <- ggplot(df_plot_genes_avg_sd, aes(x = Time, y = avg)) +
  geom_line() +
  geom_point(size=1) +
  geom_errorbar(aes(ymin = avg-sd, ymax = avg+sd), width = 3, size = 0.5) +
  labs(x = 'Time after dox induction (hours)', y = 'Average TPM') +
  ggtitle('Average TPM across Time Points') +
  facet_wrap(~ gene_name, scales = "free") +
  # Customize x-axis ticks to show only specific time points
  scale_x_continuous(breaks = c(0, 12, 24, 48, 96), labels = c("0", "12", "24", "48", "96")) +
  theme_minimal() +
  theme(
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.background = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, size = 1)
  )
ggsave("results/tpm_prolonged_genes.png", width = 20, height = 12, dpi = 500)

```

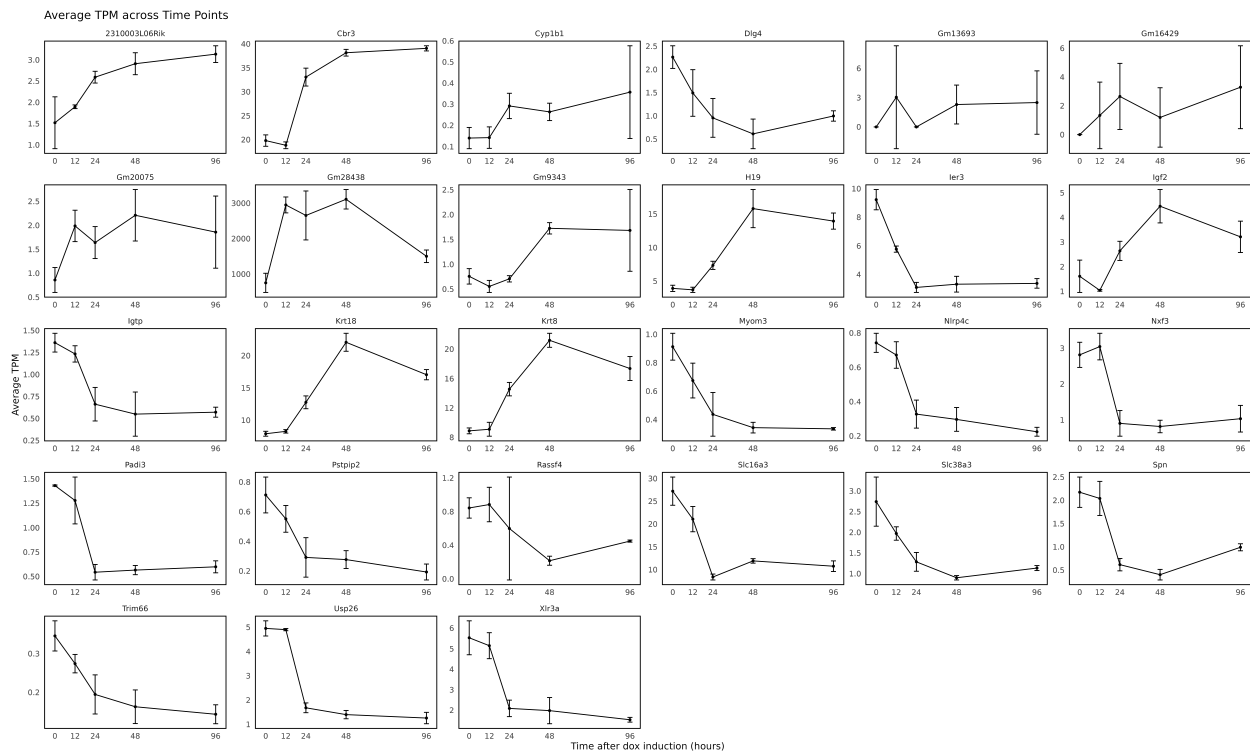


Figure 4: lfc_prolonged_genes

Heatmaps

All genes

```

# Log-transform the TPM values.
log_tpm_matrix <- log2(tpm_filtered + 1)

```

```

# Using pheatmap to plot the heatmap of the significant genes

# Add gene_names and select names from all genes list (g2s)
log_tpm_matrix_gene_names <- log_tpm_matrix %>% rownames_to_column("gene_id") %>% merge(g2s)

# Extract the only column from filtered_genes as a vector for easier manipulation
filtered_genes_vector <- filtered_genes[[1]]

# Filter to significant genes object called sig_log_tpm_matrix
sig_log_tpm_matrix <- log_tpm_matrix_gene_names[log_tpm_matrix_gene_names$gene_name %in% filtered_genes_vector]

# Remove all observations with missing data (NA)
sig_log_tpm_matrix <- na.omit(sig_log_tpm_matrix)

# Remove non numeric data for plotting heatmap
sig_log_tpm_matrix_numeric <- sig_log_tpm_matrix %>% select(-gene_id, -gene_name)

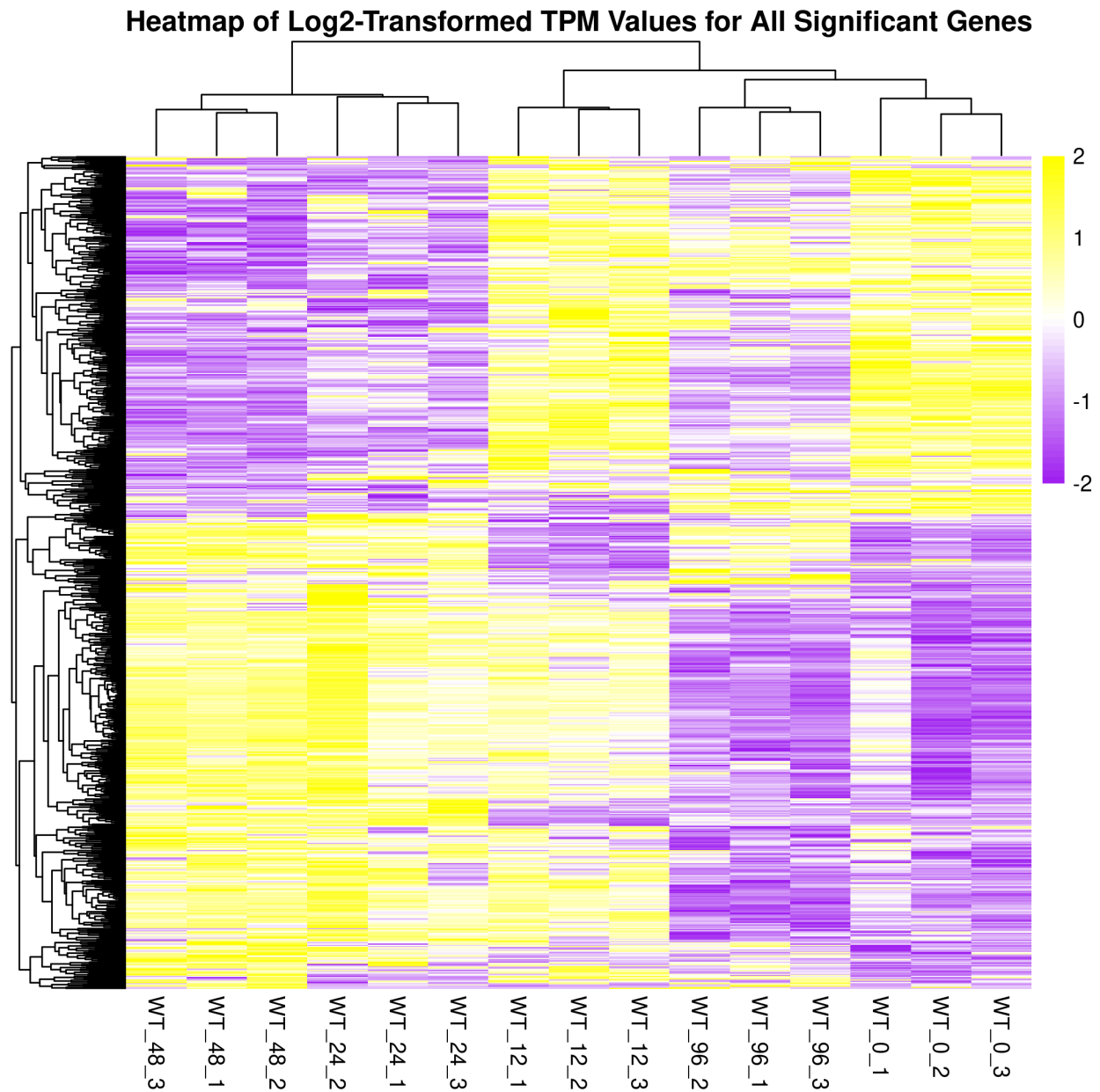
# Setting custom clustering and distance functions
custom_dist <- function(x) dist(x, method = "euclidian")
custom_hclust <- function(x) hclust(x, method = "ward.D2")

# Set the scale
breaks <- seq(-2, 2, length.out = 51)

# Plot heatmap for log transformed significant genes using custom clustering and distance methods, and
pheatmap(sig_log_tpm_matrix_numeric,
  cluster_rows = TRUE,
  cluster_cols = TRUE,
  scale = "row",
  show_rownames = FALSE,
  show_colnames = TRUE,
  main = "Heatmap of Log2-Transformed TPM Values for All Significant Genes",
  distfun = custom_dist,
  color = colorRampPalette(c("purple", "white", "yellow"))(50),
  hclustfun = custom_hclust,
  breaks = breaks,
  filename = "results/heatmap_all_genes.png"
)

save(log_tpm_matrix_gene_names, sig_log_tpm_matrix, sig_log_tpm_matrix_numeric, filtered_genes_vector,

```



Heatmap of log2 transformed tpm values for all significant genes. Note the gene expression patterns of the sample clusters formed. A total of 794 genes were significantly down or upregulated in mice after Dox treatment at any given time point. Note that groups are formed between replicates of the same time point. Samples at 0 and 96 hours form a sister group, suggesting similarities in expression patterns compared to other time point samples.

Prolonged genes

```
# Extract prolonged gene list from previously log transformed and filtered data
prolonged_sig_genes <- sig_log_tpm_matrix[sig_log_tpm_matrix$gene_name %in% genes_of_interest$gene_name]

# Remove all observations with missing data (NA)
prolonged_sig_genes <- na.omit(prolonged_sig_genes)
```

```

# Remove non numeric data for plotting heatmap
prolonged_sig_genes_numeric <- prolonged_sig_genes %>% select(-gene_id, -gene_name)

# Set rownames as the gene names to be able to visualize them in the heatmap
rownames(prolonged_sig_genes_numeric) <- prolonged_sig_genes$gene_name

# Setting custom clustering and distance functions
custom_dist <- function(x) dist(x, method = "euclidian")
custom_hclust <- function(x) hclust(x, method = "ward.D2")

# Set the scale
breaks <- seq(-2, 2, length.out = 51)

# Plot heatmap for log transformed significant genes using custom clustering and distance methods, and
pheatmap(prolonged_sig_genes_numeric,
          cluster_rows = TRUE,
          cluster_cols = TRUE,
          scale = "row",
          show_rownames = TRUE,
          show_colnames = TRUE,
          main = "Heatmap of Log2-Transformed TPM Values of Prolonged Genes",
          distfun = custom_dist,
          color = colorRampPalette(c("purple", "white", "yellow"))(50),
          hclustfun = custom_hclust,
          breaks = breaks,
          filename = "results/heatmap_prolonged_genes.png"
)

```

Heatmap of log2 transformed tpm values of prolonged genes. A total of 27 genes had a prolonged expression trend up until last sampled time point of 96 hours after Dox treatment. Note that in this case, time points 0 and 12 grouped together, suggesting similarity in expression in initial time points. However, as time point increases, samples form a different group, with samples from 24, 48 and 96 hours.

Circos plot

```

load("results/data_files.RData")
load("results/results_counts.RData")

# --- Prep Annotations ---
# Get gene annotations from gtf file and filter based on genes
gtf_file <- "/scratch/Shares/rinnclass/MASTER_CLASS/GENOMES/M25/genencode.vM25.annotation.gtf"
gtf_data <- rtracklayer::import(gtf_file)
gene_locations <- as.data.frame(gtf_data) %>%
  filter(type == "gene") %>%
  dplyr::select(chr = seqnames, start, end, gene_id, gene_name) %>%
  mutate(chr = as.character(chr))
# Check for 'chr' prefix
if (!all(str_detect(gene_locations$chr, "^chr"))) {
  gene_locations$chr <- paste0("chr", gene_locations$chr)
}
# Filter M and non-standard chromosomes

```

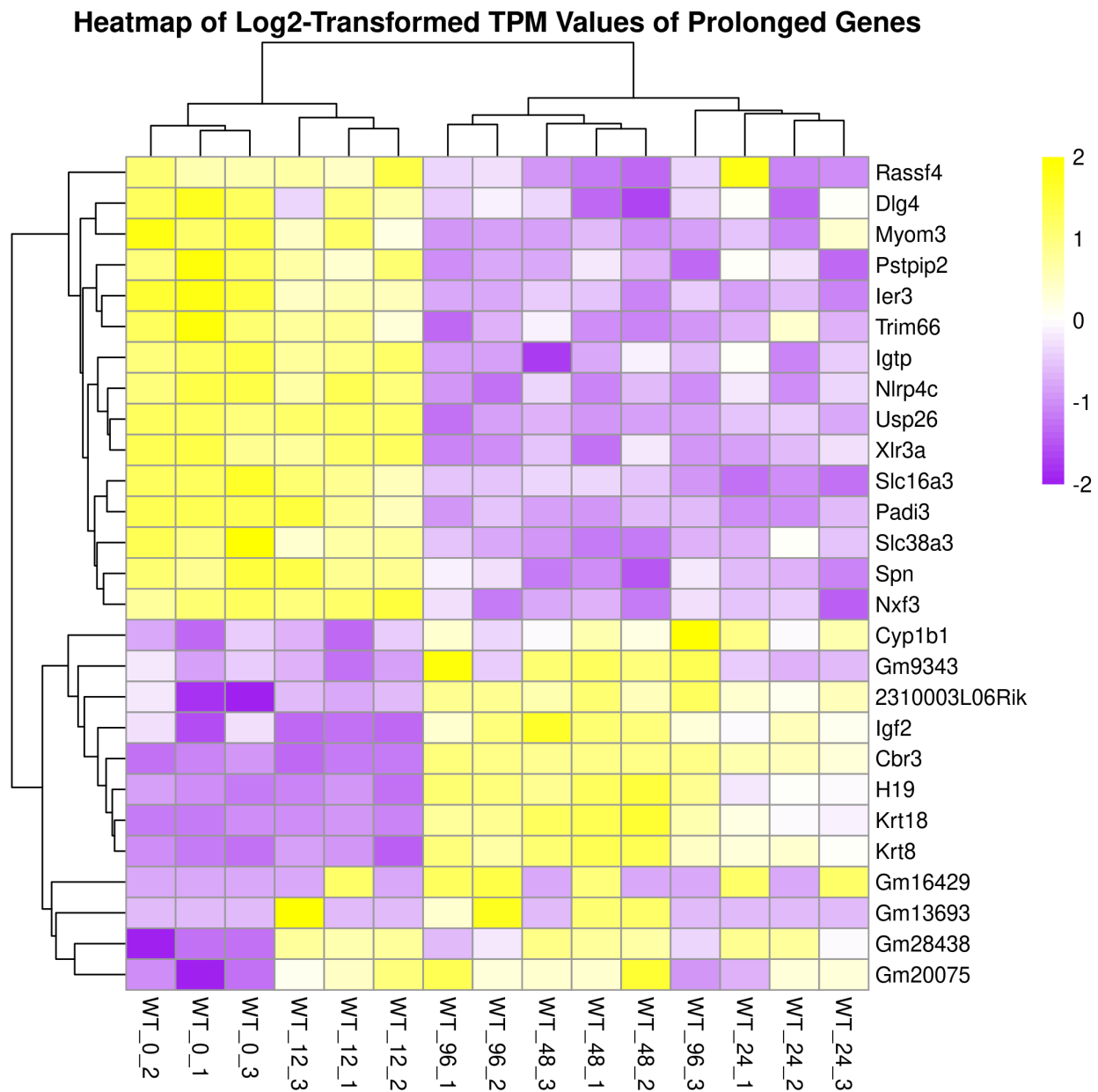



Figure 5: heatmap_prolonged_genes

```

valid_chromosomes <- paste0("chr", c(1:19, "X", "Y")) # Specific for mouse
gene_locations <- filter(gene_locations, chr %in% valid_chromosomes)
# Remove versions of gene annotations. They can cause problems
gene_locations <- gene_locations %>%
  mutate(gene_id_base = str_remove(gene_id, "\\d+"))

# --- Merge Results and Annotations ---
# Remove versions from results
de_results_prepared <- filtered_results %>%
  mutate(gene_id_base = str_remove(gene_id, "\\d+"))
# Merge DE results with gene locations based on genes found in both datasets (inner_join)
annotated_results <- inner_join(de_results_prepared, gene_locations, by = "gene_id_base")
# Selecting and renaming relevant columns for plotting and clarity
annotated_results <- annotated_results %>%
  dplyr::select(gene_id = gene_id.x, log2FoldChange, padj, gene_name = gene_name.x,
    comparison, chr, start, end)

# --- Prepare Data for Circlize ---
# Extract all unique comparisons and select for only baseline
time_points <- unique(annotated_results$comparison)
time_points <- time_points[1:4]
# Define function to fix chromosome order (it was giving bugs)
create_sorted_chr_factor <- function() {
  # Create a correctly ordered factor for chromosomes
  chr_levels <- c(paste0("chr", 1:19), "chrX", "chrY")
  return(factor(chr_levels, levels = chr_levels))
}

# --- Make Circos Plot ---
# Set up
png("results/circos_differential_expression.png", width = 3000, height = 3000, res = 300)
circos.clear()
sorted_chrs <- create_sorted_chr_factor() # Put chromosomes in order
circos.par(start.degree = 90, gap.degree = 2) # Overall parameters
circos.initializeWithIdeogram(
  species = "mm10",
  chromosome.index = levels(sorted_chrs)
)
point_cex <- 0.6 # Track parameters
track_height <- 0.15

# Find the global min and max log2FC values
global_max_fc <- max(abs(annotated_results$log2FoldChange), na.rm = TRUE)
message(paste("Original max log2FC:", global_max_fc))

# Add 20% margin to prevent points from being out of bounds
global_max_fc <- global_max_fc * 1.2
message(paste("Adjusted max log2FC with 20% margin:", global_max_fc))

# Define color mapping function for log2FoldChange values (like in a heatmap)
color_cap <- 2
get_color <- function(value) {
  if (is.na(value)) return("gray80")

```

```

capped_value <- max(min(value, color_cap), -color_cap) # cap color of outliers
intensity <- abs(capped_value) / color_cap
intensity <- sqrt(intensity) * 0.8
if (capped_value < 0) {
  return(rgb(1 - intensity * 0.7, 1 - intensity * 0.7, 1)) # More subtle blue
} else if (capped_value > 0) {
  return(rgb(1, 1 - intensity * 0.7, 1 - intensity * 0.7)) # More subtle red
} else {
  return("white") # Zero value
}
}

# Function to plot each comparison
plot_timepoint_comparison <- function(comparison_data, track_title) {
  if(nrow(comparison_data) > 0) {
    ylim <- c(-20, 20) # Global min/max for y-axis to keep scales consistent.
    circos.genomicTrack(
      comparison_data,
      ylim = ylim,
      track.height = track_height,
      bg.border = "black",
      panel.fun = function(region, value, ...) {
        # For each genomic region in this cell/sector
        for (i in 1:nrow(region)) {
          # Get log2FC for this specific point
          log2fc <- value$log2FoldChange[i]
          # Check if point is an extreme outlier
          if (abs(log2fc) > global_max_fc) {
            # Print information about extreme outliers (for debugging)
            message(sprintf("Extreme value: %s, chr=%s, log2FC=%.2f",
                          track_title, CELL_META$sector.index, log2fc))
          }
          # Get color for this specific point using the color function
          point_color <- get_color(log2fc)
          # Plot the individual point with its specific color
          circos.points(
            mean(c(region$start[i], region$end[i])), # x-position (center of region)
              log2fc, # y-position (log2FC value)
              col = point_color, # color based on log2FC
              pch = 16, # solid circle
              cex = point_cex # point size
            )
          }
        }

        # Add horizontal line at y=0
        circos.lines(CELL_META$xlim, c(0, 0), col = "black", lty = 2)
        # Add track title on the first sector
        if (CELL_META$sector.index == levels(sorted_chrs)[1]) {
          circos.text(
            CELL_META$xcenter,
            max(ylim) * 0.8, # Position text inside the track area to avoid conflicts
            track_title,
            facing = "inside",

```

```

        niceFacing = TRUE,
        adj = c(0.5, 0),
        cex = 0.8
    )
  }
}
)
} else {
  message(paste("No data for comparison:", track_title))
}
}

# Plot each timepoint comparison in separate tracks
for (time_point in time_points) {
  # Filter data for this comparison
  comparison_data <- annotated_results %>%
    filter(comparison == time_point) %>%
    dplyr::select(chr, start, end, log2FoldChange)

  # Plot this timepoint comparison
  plot_timepoint_comparison(comparison_data, time_point)
}

# Plot layout
title(main = "Differential Expression Across Time Points", cex.main = 1.2)
par(xpd = TRUE)
legend_values <- seq(-color_cap, color_cap, length.out = 7)
legend_x <- 0.9
legend_y <- -1.3
color_legend_width <- 0.2
color_legend_height <- 0.02

# Draw color bar segments
for (i in 1:(length(legend_values)-1)) {
  # Calculate color for this segment
  mid_val <- (legend_values[i] + legend_values[i+1]) / 2
  segment_color <- get_color(mid_val)

  rect(
    legend_x + (i-1) * color_legend_width/(length(legend_values)-1),
    legend_y - color_legend_height,
    legend_x + i * color_legend_width/(length(legend_values)-1),
    legend_y,
    col = segment_color,
    border = NA
  )
}

# Add tick marks and labels
for (i in 1:length(legend_values)) {
  text(
    legend_x + (i-1) * color_legend_width/(length(legend_values)-1),
    legend_y - color_legend_height*1.5,

```

```

    labels = sprintf("%.1f", legend_values[i]),
    cex = 0.7,
    adj = c(0.5, 0.5)
  )
}

# Add legend title
text(
  legend_x + color_legend_width/2,
  legend_y + color_legend_height*1.5,
  "log2FoldChange",
  cex = 0.8
)
dev.off()

## pdf
## 2

```

Results

After performing DESeq2, we found that from the 55401 total genes, 794 were significantly expressed genes. From our significantly expressed genes, we generated a list of prolonged genes that represent candidates that were up or down regulated after Dox treatment at an early time point (12 or 24 hours) and maintained their trend until the 96 hour time point (last collected). A total of 27 prolonged genes were found. We looked at prolonged up or down regulated genes, because if their levels are high or low throughout all time points, they might be causing a significant change in important genes.

IGV

We explored prolonged genes roles in the cell by looking at their gene ontologies (GO), and we found three genes with strong evidence of important regulatory roles in mice, H19, Usp26, and Ier3. From those three genes H19 had prolonged up regulation, while Ier3 and Usp26 had prolonged down regulation. Their prolonged change in expression suggests that these genes might be changing permanently, compromising cell integrity by changing regulatory pathways. However, additional time points should be sampled to test this hypothesis.

H19

This gene is a long non-coding RNA (lncRNA) gene. Strong evidence suggests that this gene is involved in the control of embryonic growth. In addition, it has been found to regulate nine genes of an imprinted gene network (Monnier et al. 2013). Its prolonged up regulation after Dox treatment suggests a potential role in maintaining cellular processes related to growth and development, possibly by modulating the activity of imprinted genes that are critical for cell differentiation and tissue patterning.

Differential Expression Across Time Points

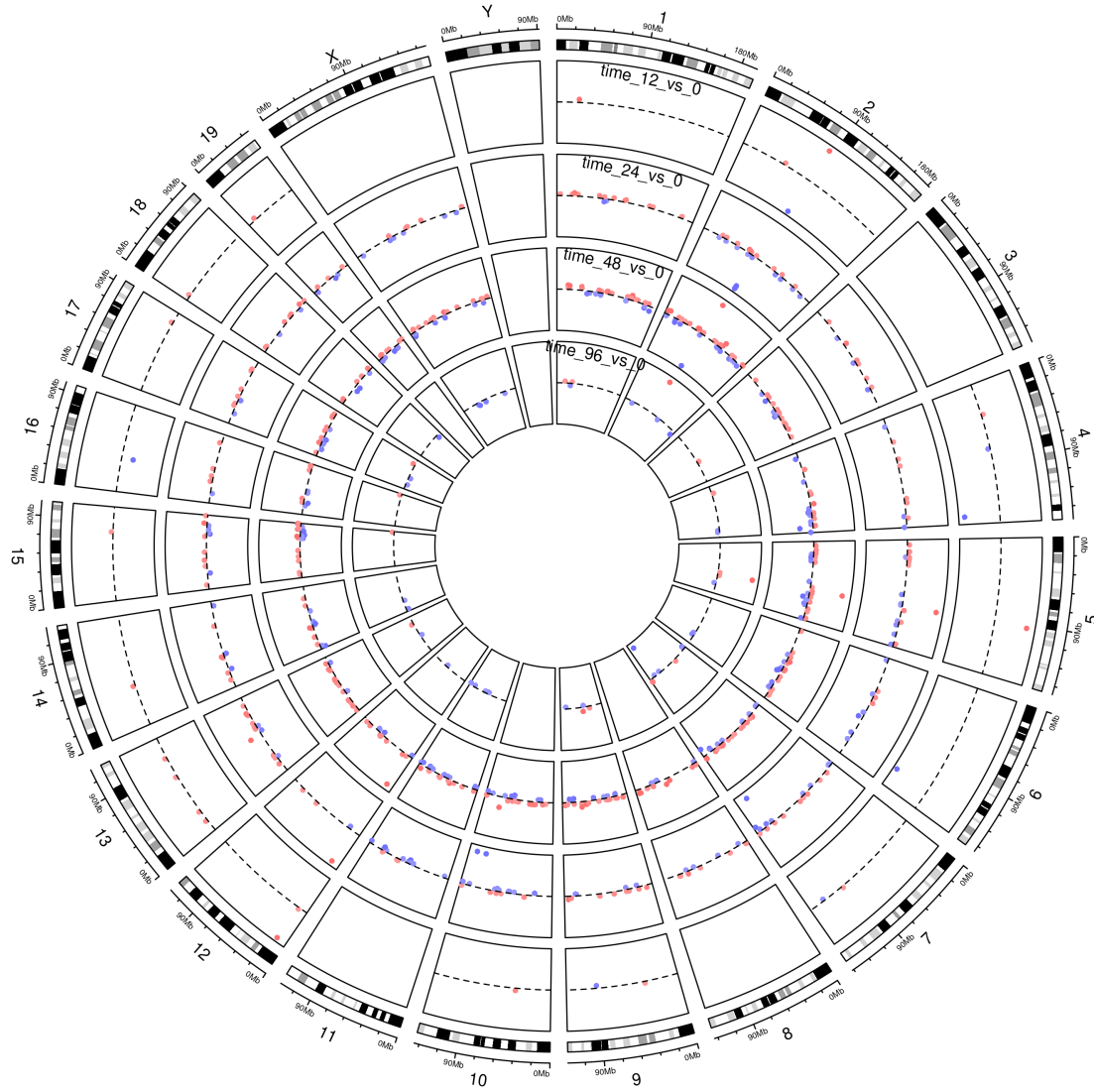
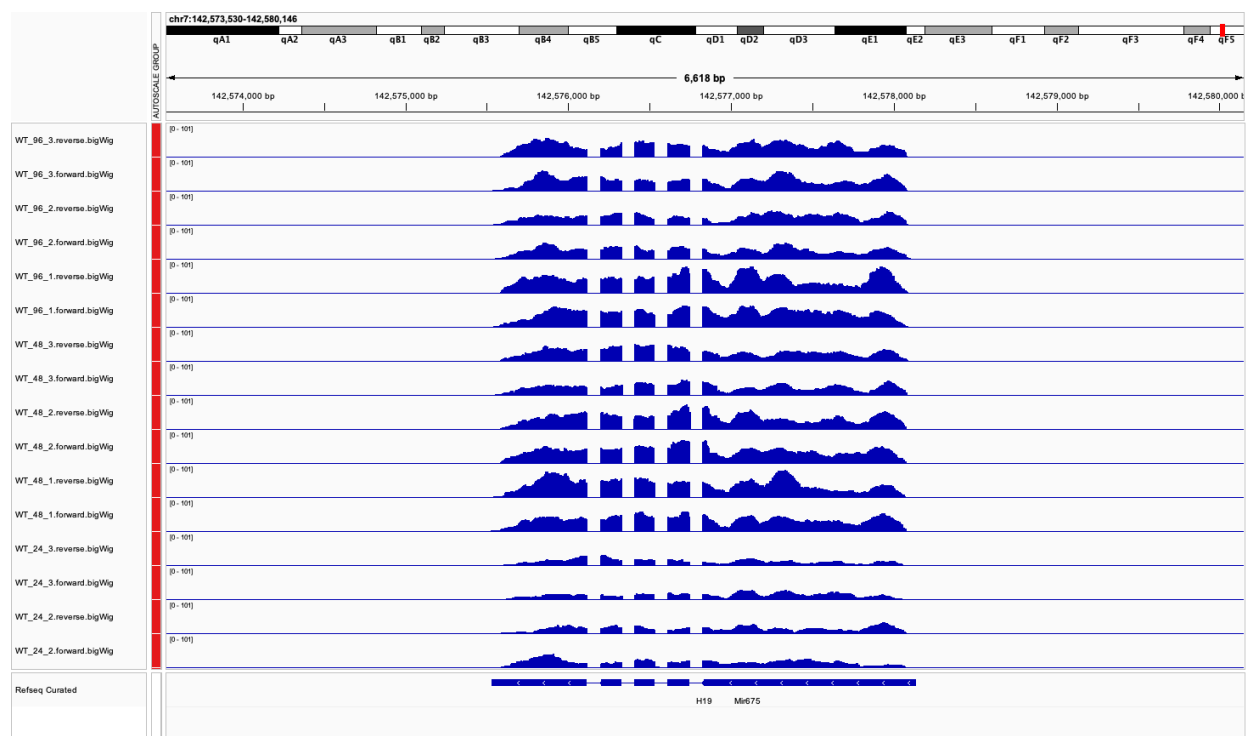


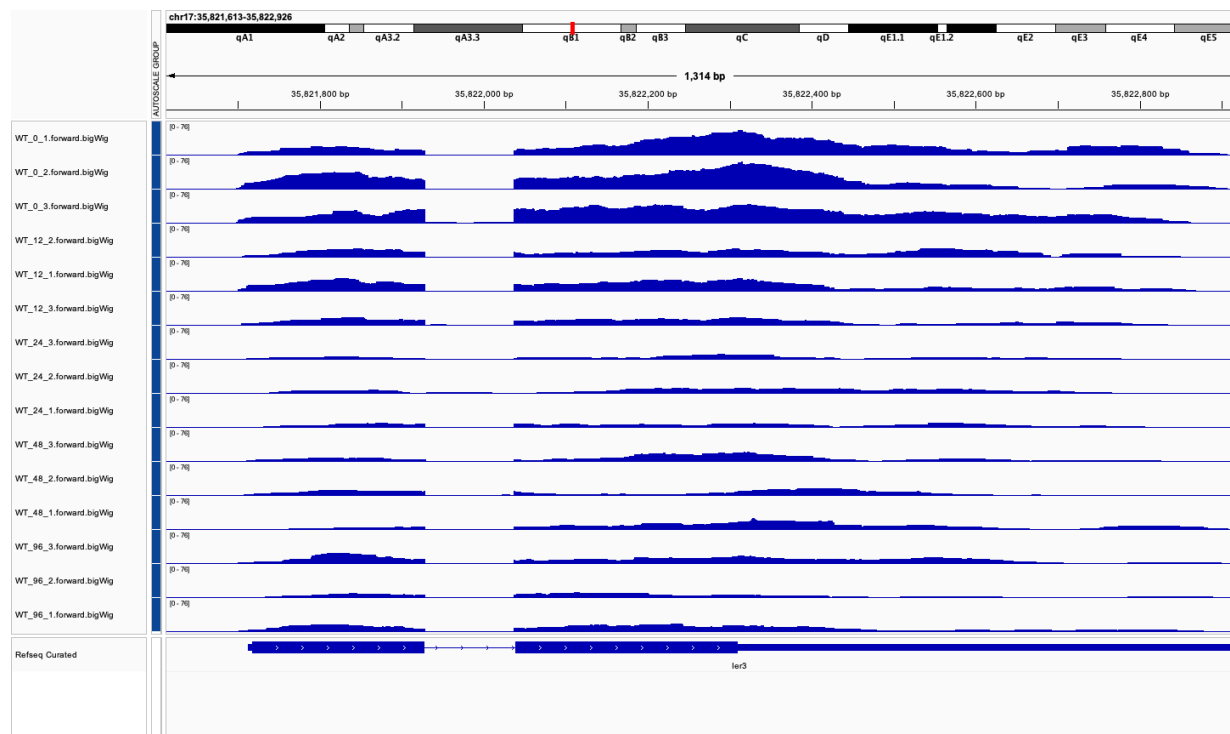
Figure 6: Circos plot



IGV track with peaks of H19 gene. Peaks represent the coverage of sequencing reads across this gene. Note that peaks are low at 0 and 12 hours, but start getting higher at 24. By the time they have reached 48 hours, they are high and they keep this high trend until the last sampled time point of 96 hours.

Ier3

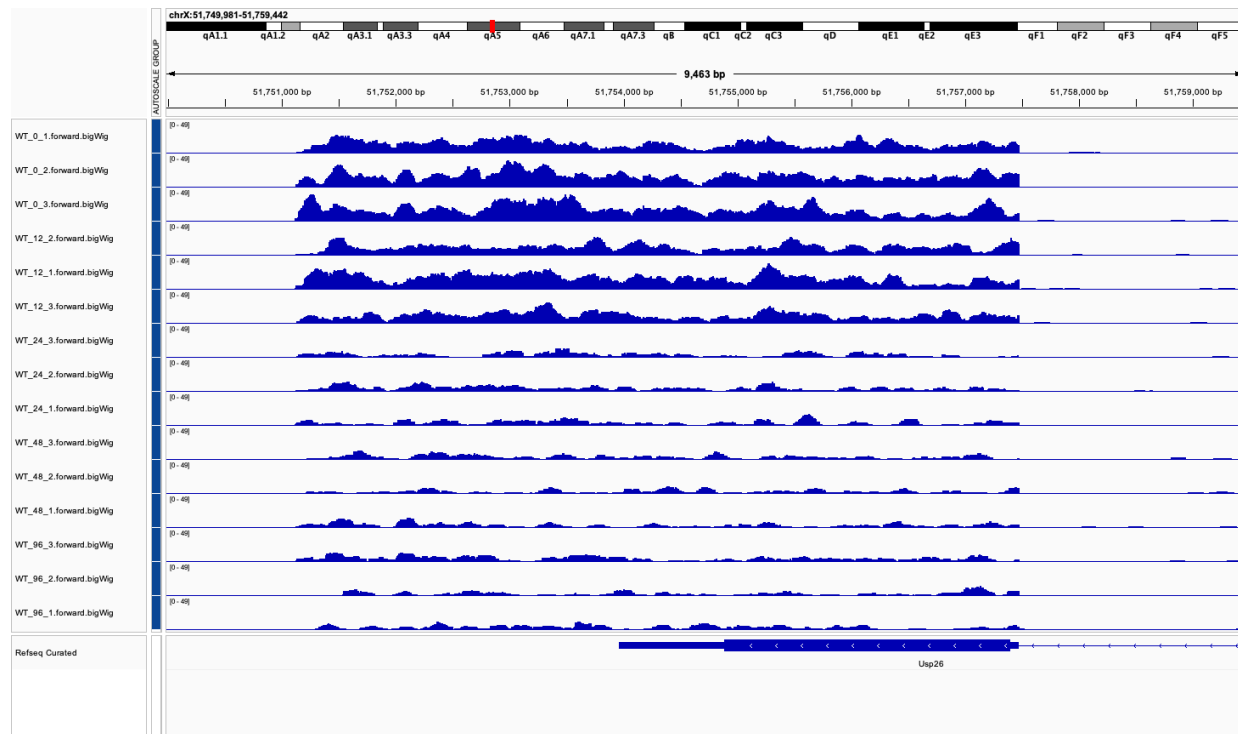
The immediate early response (Ier3) gene may play a role in the ERK pathway for multiple cellular processes, such as growth, proliferation, differentiation, and survival. This role suggests relevance in gene regulation, making it an interesting and possibly important gene to further explore the effects of Dox treatment. Dox might be affecting the normal function of this gene by down regulating its expression at least 96 hours after treatment.



IGV track with peaks of *Ier3* gene. Peaks represent the coverage of sequencing reads across this gene. Note that peaks are higher in the beginning at 0 hours, and they decrease as time increases. They remain low until the last time point sampled of 96 hours.

Usp26

The *Usp26* gene has been involved in deubiquitination pathways, making it potentially relevant for gene regulation. There are many transcription factors in the cell, such as polycomb repressive complex 1 (PRC1), that are in charge of maintaining and regulating epigenetic marks in the genome (Tamburri et al. 2020). Many of these transcription factors, need ubiquitination for their function, making a deubiquitination crucial for this process as well (Mark and Rape 2021). For example, this gene has been found to be involved in somatic cell reprogramming through the K48 deubiquitination of two protein components of PRC1.



IGV track with peaks of *Usp26* gene. Peaks represent the coverage of sequencing reads across this gene. Note that peaks are higher at 0 and 12 hours, but are lower when 24 hours are reached. They keep being low until the last time point of 96 hours.

Bibliography

- Mark, Kevin G, and Michael Rape. 2021. "Ubiquitin-dependent Regulation of Transcription in Development and Disease." *EMBO Reports* 22 (4): e51078. <https://doi.org/10.15252/embr.202051078>.
- Monnier, Paul, Clémence Martinet, Julien Pontis, Irina Stancheva, Slimane Ait-Si-Ali, and Luisa Dandolo. 2013. "H19 lncRNA Controls Gene Expression of the Imprinted Gene Network by Recruiting MBD1." *Proceedings of the National Academy of Sciences* 110 (51): 20693–98. <https://doi.org/10.1073/pnas.1310201110>.
- Tamburri, Simone, Elisa Lavarone, Daniel Fernández-Pérez, Eric Conway, Marika Zanotti, Daria Manganaro, and Diego Pasini. 2020. "Histone H2AK119 Mono-Ubiquitination Is Essential for Polycomb-Mediated Transcriptional Repression." *Molecular Cell* 77 (4): 840–856.e5. <https://doi.org/10.1016/j.molcel.2019.11.021>.